

"Coding Standards & Best Practices"

Document Type:	Development Rules & Guidelines
Version:	v1.0
Date:	June 2026
Status:	Mandatory for All Contributors

1. Naming Conventions

Category	Rule	Example
Components	PascalCase, descriptive	IdeaCard, EvolutionTree, BlueprintTab
Hooks	camelCase, use prefix	useIdeas, useEvolution, useAuth
Utils	camelCase, verb-noun	generateIdeas, parseInput, scoreIdea
Types/Interfaces	PascalCase, no I prefix	UserProfile, IdeaCardProps, BlueprintData
Constants	UPPER_SNAKE_CASE	MAX_IDEAS, DEFAULT_TIMEOUT
API Routes	kebab-case	/api/generate-ideas, /api/evolve-idea
Database Tables	PascalCase, singular	User, Project, Idea, Blueprint
Environment Variables	UPPER_SNAKE_CASE	OPENAI_API_KEY, DATABASE_URL
CSS Classes	kebab-case	idea-card, evolution-tree, blueprint-section
File Names	kebab-case	idea-generator.tsx, evolution-engine.ts

2. Code Structure Rules

Single Responsibility

Each component/function does ONE thing. Max 200 lines per file.

Folder Organization

Components by feature, not by type. Shared components in /components/ui/.

Import Order

1. React/Next, 2. Third-party, 3. Absolute imports (@/), 4. Relative imports (./).

Export Pattern

Named exports for utilities. Default exports for pages. Barrel exports for index files.

Type Safety

No 'any' type. Strict TypeScript. All API responses typed.

Error Handling

Try-catch for all async operations. Custom error classes. User-friendly messages.

Comments

WHY not WHAT. Complex logic needs explanation. No commented-out code.

Magic Numbers

No hardcoded numbers. Use named constants.

3. AI Agent Development Rules

Temperature Guidelines

Parser/Critic/Scorer: 0.2-0.3 (objective). Generator/Evolver/Designer: 0.5-0.6 (creative). Never exceed 0.8.

Prompt Engineering

System prompts must include: Role definition, Input format, Output format, Rules/Constraints, Examples.

Output Parsing

Always use StructuredOutputParser or Zod schemas. Never trust raw LLM output.

Error Fallbacks

If AI fails, return cached response or generic fallback. Never crash the app.

Cost Tracking

Log every API call: tokens used, cost, latency. Alert if daily budget exceeds 80%.

Rate Limiting

Respect OpenAI rate limits. Implement exponential backoff. Queue heavy requests.

Context Window

Monitor token usage. Summarize long contexts. Split into chunks if needed.

Caching Strategy

Cache identical prompts for 24h. Cache embeddings indefinitely.

4. Database Rules

Schema Changes

All changes via Prisma migrations. Never modify DB directly.

Query Optimization

Use SELECT only needed fields. Avoid N+1 queries. Use includes carefully.

Indexing

Index all foreign keys. Index frequently queried fields. GIN index for JSONB.

Data Integrity

Foreign key constraints. NOT NULL where appropriate. Default values.

Soft Deletes

Never hard delete user data. Use status field (ACTIVE, ARCHIVED, DELETED).

Migrations

Test migrations on staging before production. Always have rollback plan.

Connection Pooling

Max 20 connections per instance. Use PgBouncer for serverless.

Backup

Daily automated backups. 30-day retention. Test restore monthly.

5. API Development Rules

RESTful Design

Nouns for resources, verbs for methods. `/api/projects` not `/api/getProjects`.

Status Codes

200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 429 Rate Limited, 500 Server Error.

Response Format

Consistent JSON: `{ success: boolean, data?: T, error?: string, meta?: object }`.

Validation

Zod schemas for ALL request bodies. Validate before processing.

Authentication

Check auth on every protected route. Return 401 if missing/invalid.

Authorization

Check ownership. User A cannot access User B's data. Return 403 if unauthorized.

Rate Limiting

Per-user + per-IP limits. Return 429 with `Retry-After` header.

Logging

Log all requests: `userId`, `endpoint`, `status`, `duration`, `timestamp`.

Error Handling

Don't leak sensitive info in errors. Log details server-side.

Pagination

All list endpoints paginated. Default: `page=1`, `limit=20`, `max=100`.

6. UI/UX Development Rules

Responsive First

Mobile-first design. All features work on 320px+.

Accessibility

WCAG 2.1 AA. Alt text for images. ARIA labels. Keyboard navigation. Focus states.

Loading States

Never leave user waiting. Skeletons, spinners, progress bars.

Error States

Friendly error messages. Retry buttons. Fallback content.

Empty States

Don't show blank screens. Helpful illustrations + CTAs.

Animations

Subtle, purposeful. Max 300ms duration. Respect prefers-reduced-motion.

Color Contrast

Minimum 4.5:1 for normal text. 3:1 for large text.

Touch Targets

Minimum 44x44px for mobile. Adequate spacing between elements.

Dark Mode

Default dark theme. Consistent color tokens. No hardcoded colors.

Component Reusability

Build generic components. Props for customization. Document in Storybook.

7. Testing Rules

Coverage Target

Unit tests: 80%+ coverage. Integration: All API endpoints. E2E: Critical flows.

Test Naming

describe('Feature') + it('should do X when Y'). Clear and descriptive.

Mocking

Mock external APIs (OpenAI, GitHub). Mock database for unit tests.

Test Data

Use factories, not fixtures. Clean up after each test.

Flaky Tests

Zero tolerance. Retry mechanism for async operations. Stable selectors.

Performance Tests

Lighthouse CI on every PR. Bundle size monitoring.

Security Tests

Dependency audit weekly. Penetration test before major releases.

8. Git & Collaboration Rules

Branch Naming

feature/IDEA-xxx-description, bugfix/BUG-xxx-description, hotfix/critical-fix.

Commit Messages

Conventional Commits: feat:, fix:, docs:, refactor:, test:, chore:. Imperative mood.

Pull Requests

Template: What, Why, How. Screenshots for UI changes. Link to ticket.

Code Review

All PRs need 1 approval. No self-merge. Address all comments before merge.

Merge Strategy

Squash and merge for features. Rebase for long-lived branches.

CI/CD

All checks must pass: lint, test, build, type-check. No bypass.

Main Branch

Always deployable. Tag releases. Semantic versioning (v1.0.0).

9. Security Rules

Never Commit Secrets

API keys, passwords, tokens in .env only. .env in .gitignore.

Input Sanitization

All user inputs sanitized. XSS prevention. SQL injection prevention.

Authentication

JWT with expiry. Refresh token rotation. Secure cookie flags.

Authorization

RBAC on every route. Principle of least privilege.

Data Encryption

AES-256 at rest. TLS 1.3 in transit. No plaintext storage.

Dependency Management

npm audit weekly. Snyk integration. Pin versions.

Error Messages

Generic to user. Detailed logs server-side only.

CORS

Whitelist only. No wildcard in production.

10. Performance Rules

Bundle Size

Max 200KB initial JS. Code splitting per route. Tree shaking.

Images

WebP format. Lazy loading. Responsive sizes. Next.js Image component.

API Calls

Batch requests. Debounce user input. Cache aggressively.

Database

Connection pooling. Query optimization. Read replicas for reads.

Lazy Loading

Heavy components loaded on demand. Blueprint/Mockup only on selection.

Memory

No memory leaks. Cleanup subscriptions. Limit concurrent requests.

Core Web Vitals

LCP < 2.5s, FID < 100ms, CLS < 0.1. Monitor continuously.

— End of Development Rules & Guidelines —